Week 5 Lab Handout

API Build I — From Contract to Running Code

Time: 110 minutes **Bring: laptop, openapi.yaml from Week 4, Swagger UI open**

Before you start: your contract just became your test suite

Everything you build today gets checked against one thing: does it actually return what `openapi.yaml` promised? Not "close enough," not "works when I test it once by hand" — the exact field names, the exact types, the exact status codes you wrote down last week.

> **Why this matters:** your ring partner never reads your code. They only ever see what your API actually returns. A response that "looks right" to you but silently drifts from the contract breaks their code while yours keeps "working fine" in isolation. That gap is invisible until someone hits it — which is exactly why today's habit is **verify, don't eyeball.**

Part A — Quick Setup (10 min)

1. Confirm your development server runs and you can reach it locally.
2. Open your `openapi.yaml` from Week 4 and your Swagger UI (or Swagger Editor's preview) side by side.
3. List every **GET** endpoint in your contract — these, and only these, are today's scope. Writes (POST/PUT/PATCH/DELETE) are next week.

Part B — Implement Your GET Endpoints (45 min)

For every GET endpoint in your contract, wire up the full path: request comes in, your route handler runs, it fetches real data, and the response is shaped to match your schema exactly before it goes out.

The step that actually matters: shaping the response

Your database almost never returns data in the exact shape your contract promises — different field names, different types, extra internal columns. That's expected. What's not optional is the **mapping step** between what your database gives you and what your API sends back:

```
Database gives you: { "_id": 5, "title": "Ship v1", "due_date": "2026-08-13T00:00:00.000Z" }
Contract promises: { id: integer, title: string, dueDate: string (date-time) }
Your response sends: { "id": 5, "title": "Ship v1", "dueDate": "2026-08-13T00:00:00Z" }
```

Same data. One extra step. Skipping it is where almost every drift bug in this course starts.

As you build, keep asking

- Does this field's name match the contract exactly — not close, exactly?
- Is the type what the contract says (a real date string, not a timestamp; a number, not a numeric string)?
- Am I returning the right status code for every case this endpoint can hit — not just the happy path?

Part C — Verify Against Your Contract (20 min)

For **every** endpoint you finish, do this before moving to the next one — not once at the end for everything:

1. Open the endpoint in Swagger UI and click **"Try it out."**

2. Send a real request.
3. Compare the real response to your own schema, **field by field** — not from memory, side by side on screen.

This takes about 30 seconds per endpoint and catches what a glance never will. An endpoint that “looks right” and an endpoint that's actually verified are not the same thing, and only one of them is safe to hand to a partner.

What you're specifically checking for

- [ ] Every field in the response has the exact name the contract promised
- [ ] Every field's type matches (string vs. number vs. boolean vs. date-time)
- [ ] No extra fields the contract didn't mention (especially internal IDs or DB-only columns)
- [ ] No missing fields the contract marked as required
- [ ] The status code matches what the contract says for this case (200 for success, 404 for not found, etc.)

Part D — Fix Any Drift You Find (20 min)

Go back through what Part C surfaced. For each mismatch, you have exactly two honest options:

• **Fix the implementation.** This is the right call almost every time — the contract was correct, your code wasn't matching it yet.

• **Change the contract.** Sometimes building the real thing reveals the contract itself was wrong (a field that should've been optional, a type that doesn't fit reality). If this happens, update `openapi.yaml` — but don't do it silently.

Part E — Finalize (15 min)

Create `CONTRACT_DEVIATIONS.md` at your repo root. If you changed anything in `openapi.yaml` during Part D, document it here: what changed, and why the original contract didn't hold up. If nothing changed, say so explicitly — “no deviations” is a valid and expected entry, not a gap.

Commit and push:

```
git add . CONTRACT_DEVIATIONS.md
git commit -m "Week 5: GET endpoints implemented and verified against contract"
git push
```

(Still no branch-naming convention to follow — that starts once you're working out of the template repository your instructor hands you.)

This week's deliverable checklist

- [ ] Every GET endpoint from openapi.yaml is implemented and running
- [ ] Every response verified against the contract field-by-field in Swagger UI, not by eye
- [ ] Any drift found was fixed, not left in place
- [ ] Status codes are correct for both success and not-found cases
- [ ] CONTRACT_DEVIATIONS.md created, documenting any real changes (or confirming none)
- [ ] All code committed and pushed

Rubric (35 pts)

Criterion	Points
All GET endpoints implemented and running	10
Responses match contract schema exactly (field names, types, structure)	12
Correct status codes for success and not-found cases	8
Evidence of systematic verification against the contract, not eyeballing	5

Common problems and quick fixes

Problem	Fix
Response fields don't match the contract	Check for the classic three: wrong casing (user_id vs userId), extra DB-only fields, wrong data type — add the mapping step between your database and your response
Not sure if a mismatch should be fixed in code or in the contract	Ask: was the original contract genuinely wrong, or did I just build it differently? If the contract was reasonable, fix your code, not the contract
Swagger UI “Try it out” doesn't reach your server	Confirm your server is actually running locally and Swagger UI is pointed at the right base URL
Dates keep coming out as timestamps	Almost always a database default — explicitly format the date to an ISO string before it leaves your response, don't pass the raw driver value through
You changed the contract but didn't tell anyone	Update CONTRACT_DEVIATIONS.md immediately, and flag it to your downstream ring partner directly — they built against the version you already sent them in Week 4

Next week: Build continues — POST, PUT, PATCH, and DELETE, where validation and real state changes make everything a little less forgiving than a read ever is.